



Reduced instruction set computer

In electronics and computer science, a **reduced instruction set computer** (**RISC**) (pronounced "risk") is a computer architecture designed to simplify the individual instructions given to the computer to accomplish tasks. Compared to the instructions given to a complex instruction set computer (CISC), a RISC computer might require more machine code in order to accomplish a task because the individual instructions perform simpler operations. The goal is to offset the need to process more instructions by increasing the speed of each instruction, in particular by implementing an instruction pipeline, which may be simpler to achieve given simpler instructions.^[1]

The key operational concept of the RISC computer is that each instruction performs only one function (e.g. copy a value from memory to a register). The RISC computer usually has many (16 or 32) high-speed, general-purpose registers with a load-store architecture in which the code for the register-register instructions (for performing arithmetic and tests) are separate from the instructions that access the main memory of the computer. The design of the CPU allows RISC computers few simple addressing modes^[2] and predictable instruction times that simplify design of the system as a whole.

The conceptual developments of the RISC computer architecture began with the IBM 801 project in the late 1970s, but these were not immediately put into use. Designers in California picked up the 801 concepts in two seminal projects, Stanford MIPS and Berkeley RISC. These were commercialized in the 1980s as the MIPS and SPARC systems. IBM eventually produced RISC designs based on further work on the 801 concept, the IBM POWER architecture, PowerPC, and Power ISA. As the projects matured, many similar designs, produced in the mid-to-late 1980s and early 1990s, such as ARM, PA-RISC, and Alpha, created central processing units that increased the commercial utility of the Unix workstation and of embedded processors in the laser printer, the router, and similar products.

In the minicomputer market, companies that included Celerity Computing, Pyramid Technology, and Ridge Computers began offering systems designed according to RISC or RISC-like principles in the early 1980s.^{[3][4][5][6][7]} Few of these designs began by using RISC microprocessors.

The varieties of RISC processor design include the ARC processor, the DEC Alpha, the AMD Am29000, the ARM architecture, the Atmel AVR, Blackfin, Intel i860, Intel i960, LoongArch, Motorola 88000, the MIPS architecture, PA-RISC, Power ISA, RISC-V, SuperH, and SPARC. RISC processors are used in supercomputers, such as the Fugaku.^[8]



The Sun Microsystems UltraSPARC processor is a type of RISC microprocessor.

History and development

A number of systems, going back to the 1960s, have been credited as the first RISC architecture, partly based on their use of the load-store approach.^[9] The term RISC was coined by David Patterson of the Berkeley RISC project, although somewhat similar concepts had appeared before.^[10]

The CDC 6600 designed by Seymour Cray in 1964 used a load-store architecture with only two addressing modes (register+register, and register+immediate constant) and 74 operation codes, with the basic clock cycle being 10 times faster than the memory access time.^[11] Partly due to the optimized load-store architecture of the CDC 6600, Jack Dongarra says that it can be considered a forerunner of modern RISC systems, although a number of other technical barriers needed to be overcome for the development of a modern RISC system.^[12]

IBM 801

Michael J. Flynn views the first RISC system as the IBM 801 design,^[2] begun in 1975 by John Cocke and completed in 1980. The 801 developed out of an effort to build a 24-bit high-speed processor to use as the basis for a digital telephone switch. To reach their goal of switching 1 million calls per hour (300 per second) they calculated that the CPU required performance on the order of 12 million instructions per second (MIPS),^[13] compared to their fastest mainframe machine of the time, the 370/168, which performed at 3.5 MIPS.^[14]

The design was based on a study of IBM's extensive collection of statistics gathered from their customers. This demonstrated that code in high-performance settings made extensive use of processor registers, and that they often ran out of them. This suggested that additional registers would improve performance. Additionally, they noticed that compilers generally ignored the vast majority of the available instructions, especially orthogonal addressing modes. Instead, they selected the fastest version of any given instruction and then constructed small routines using it. This suggested that the majority of instructions could be removed without affecting the resulting code. These two conclusions worked in concert; removing instructions would allow the instruction opcodes to be shorter, freeing up bits in the instruction word which could then be used to select among a larger set of registers.^[13]

The telephone switch program was canceled in 1975, but by then the team had demonstrated that the same design would offer significant performance gains running just about any code. In simulations, they showed that a compiler tuned to use registers instead of operating directly on memory would run code about three times as fast as traditional designs. Somewhat surprisingly, the same code would run about 50% faster even on existing machines due to the improved register use. In practice, their experimental PL/8 compiler, a slightly cut-down version of PL/I, consistently produced code that ran much faster on their existing mainframes.^[13]

A 32-bit version of the 801 was eventually produced in a single-chip form as the IBM ROMP in 1981, which stood for 'Research OPD [Office Products Division] Micro Processor'.^[15] This CPU was designed for "mini" tasks, and found use in peripheral interfaces and channel controllers on later IBM computers. It was also used as the CPU in the IBM RT PC in 1986, which turned out to be a commercial failure.^[16] Although the 801 did not see widespread use in its original form, it inspired many research projects, including ones at IBM that would eventually lead to the IBM POWER architecture.^{[17][18]}

Berkeley RISC and Stanford MIPS

By the late 1970s, the 801 had become well-known in the industry. This coincided with new fabrication techniques that were allowing more complex chips to come to market. The Zilog Z80 of 1976 had 8,000 transistors, whereas the 1979 Motorola 68000 (68k) had 68,000. These newer designs generally used their newfound complexity to expand the instruction set to make it more orthogonal. Most, like the 68k, used microcode to do this, reading instructions and re-implementing them as a sequence of simpler internal instructions. In the 68k, a full $\frac{1}{3}$ of the transistors were used for this microcoding.^[19]

In 1979, David Patterson was sent on a sabbatical from the University of California, Berkeley to help DEC's west-coast team improve the VAX microcode. Patterson was struck by the complexity of the coding process and concluded it was untenable.^[20] He first wrote a paper on ways to improve microcoding, but later changed his mind and decided microcode itself was the problem. With funding from the DARPA VLSI Program, Patterson started the Berkeley RISC effort. The Program, practically unknown today, led to a huge number of advances in chip design, fabrication, and even computer graphics. Considering a variety of programs from their BSD Unix variant, the Berkeley team found, as had IBM, that most programs made no use of the large variety of instructions in the 68k.^[21]

Patterson's early work pointed out an important problem with the traditional "more is better" approach; even those instructions that were critical to overall performance were being delayed by their trip through the microcode. If the microcode was removed, the programs would run faster. And since the microcode ultimately took a complex instruction and broke it into steps, there was no reason the compiler could not do this instead. These studies suggested that, even with no other changes, one could make a chip with $\frac{1}{3}$ fewer transistors that would run faster.^[21] In the original RISC-I paper they noted:^[22]

Skipping this extra level of interpretation appears to enhance performance while reducing chip size.^[22]

It was also discovered that, on microcoded implementations of certain architectures, complex operations tended to be *slower* than a sequence of simpler operations doing the same thing. This was in part an effect of the fact that many designs were rushed, with little time to optimize or tune every instruction; only those used most often were optimized, and a sequence of those instructions could be faster than a less-tuned instruction performing an equivalent operation as that sequence. One infamous example was the VAX's INDEX instruction.^[23]

The Berkeley work also turned up a number of additional points. Among these was the fact that programs spent a significant amount of time performing subroutine calls and returns, and it seemed there was the potential to improve overall performance by speeding these calls. This led the Berkeley design to select a method known as register windows, which can significantly improve subroutine performance, although at the cost of some complexity.^[22] They also noticed that the majority of mathematical instructions were simple assignments; only $\frac{1}{3}$ of them actually performed an operation like addition or subtraction. But when those operations did occur, they tended to be slow. This led to far more emphasis on the underlying arithmetic data unit, as opposed to previous designs where the majority of the chip was dedicated to control and microcode.^[21]

The resulting Berkeley RISC was based on gaining performance through the use of pipelining and aggressive use of register windowing.^{[23][22]} In a traditional CPU, one has a small number of registers, and a program can use any register at any time. In a CPU with register windows, there are a huge number of registers, e.g., 128, but programs can only use a small number of them, e.g., eight, at any one time. A program that limits itself to eight registers per procedure can make very fast procedure calls: The call simply moves the window "down" by eight, to the set of eight registers used by that procedure, and the return moves the window back.^[24] The Berkeley RISC project delivered the RISC-I processor in 1982. Consisting of only 44,420 transistors (compared with averages of about 100,000 in newer CISC designs of the era), RISC-I had only 32 instructions, and yet completely outperformed any other single-chip design, with estimated performance being higher than the VAX.^[22] They followed this up with the 40,760-transistor, 39-instruction RISC-II in 1983, which ran over three times as fast as RISC-I.^[22]

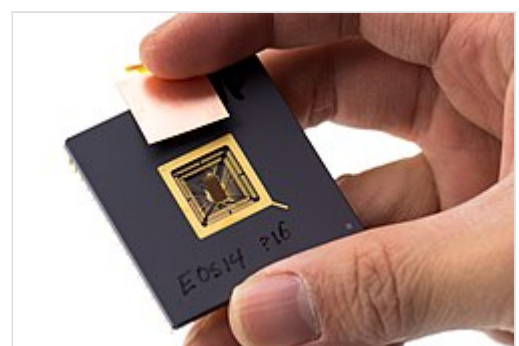
As the RISC project began to become known in Silicon Valley, a similar project began at Stanford University in 1981. This MIPS project grew out of a graduate course by John L. Hennessy, produced a functioning system in 1983, and could run simple programs by 1984.^[25] The MIPS approach emphasized an aggressive clock cycle and the use of the pipeline, making sure it could be run as "full" as possible.^[25] The MIPS system was followed by the MIPS-X, and in 1984 Hennessy and his colleagues formed MIPS Computer Systems to produce the design commercially.^{[25][26]} The venture resulted in a new architecture that was also called MIPS and the R2000 microprocessor in 1985.^[26]

The overall philosophy of the RISC concept was widely understood by the second half of the 1980s and led the designers of the MIPS-X to put it this way in 1987:

The goal of any instruction format should be: 1. simple decode, 2. simple decode, and 3. simple decode. Any attempts at improved code density at the expense of CPU performance should be ridiculed at every opportunity.^[27]

Competition between RISC and conventional CISC approaches was also the subject of theoretical analysis in the early 1980s, leading, for example, to the iron law of processor performance.

Since 2010, a new open standard instruction set architecture (ISA), Berkeley RISC-V, has been under development at the University of California, Berkeley, for research purposes and as a free alternative to proprietary ISAs. As of 2014, version 2 of the user space ISA is fixed.^[28] The ISA is designed to be extensible from a barebones core sufficient for a small embedded processor to supercomputer and cloud computing use with standard and chip designer-defined extensions and coprocessors. It has been tested in silicon design with the ROCKET SoC, which is also available as an open-source processor generator in the CHISEL language.



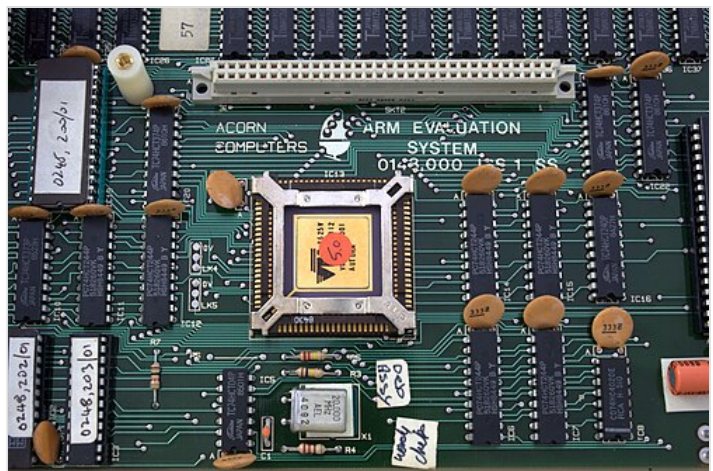
RISC-V prototype chip (2013)

Commercial breakout

In the early 1980s, significant uncertainties surrounded the RISC concept. One concern involved the use of memory; a single instruction from a traditional processor like the Motorola 68k may be written out as perhaps a half dozen of the simpler RISC instructions. In theory, this could slow the system down as it

spent more time fetching instructions from memory. But by the mid-1980s, the concepts had matured enough to be seen as commercially viable.^{[16][25]}

Commercial RISC designs began to emerge in the mid-1980s. The IBM RT PC was introduced in January 1986, and the Acorn ARM1 was introduced as part of an evaluation system in July 1986,^[29] following on from initial fabrication in April 1985.^[30] The MIPS R2000 was introduced in May 1986,^[31] followed shortly thereafter by Hewlett-Packard's PA-RISC in some of their computers.^[32] In the meantime, the Berkeley effort had become so well known that it eventually became the name for the entire concept. In 1987 Sun Microsystems began shipping systems with the SPARC processor, directly based on the Berkeley RISC-II system.^{[16][33]} The US government Committee on Innovations in Computing and Communications credits the acceptance of the viability of the RISC concept to the success of the SPARC system.^[16]



Acorn ARM Evaluation System (1985)

A benefit of RISC to the industry was much lower prices than the sole sourced Intel 80386,^[34] by 1989 many RISC CPUs were available, and competition lowered their price to \$10 per MIPS in large quantities. The performance of IBM's RISC CPU—only available in the RT PC—was less competitive than others,^[35] but the success of SPARC renewed interest within IBM, which released new RISC systems by 1990 and by 1995 RISC processors were the foundation of a \$15 billion server industry.^[16]

Patterson was a consultant for Intel during the 1980s. Despite dominating the microprocessor market with the CISC 80386 and its successors, the consensus that RISC was the future of the industry influenced that company's engineers, who worked to improve the performance of their processors to match RISC.^[34] IBM chose a similar approach of improving CISC as much as possible with IBM mainframes,^[36] while introducing new RISC-based computers that competed directly with its existing CISC midrange systems.^[37] Sun's move to SPARC was based on its decision to, instead of relying on improvements to CISC, transition to RISC as fast as possible.^[36] The DEC engineers who designed DECstation with MIPS R2000 and not their company's own VAX did so because, they wrote, the former offered "at least a two-to-one performance advantage" for the cost.^{[38][36]} By the later 1980s, new RISC designs were easily outperforming all CISC designs by a wide margin.^[36]

By 1992 Sun, DEC, HP, and IBM were the leading commercial RISC companies.^[39] Other vendors began their own RISC efforts. Among the available architectures were the DEC Alpha, AMD Am29000, Intel i860 and i960, Motorola 88000, IBM POWER, and, slightly later, the IBM/Apple/Motorola PowerPC. Many of these have since disappeared due to them often offering no competitive advantage over others of the same era. Those that remain are often used only in niche markets or as parts of other systems; of the designs from these traditional vendors, only SPARC and the PowerPC-derived Power ISA have any significant remaining market.

The ARM architecture has been the most widely adopted RISC ISA, initially intended to deliver higher-performance desktop computing, at low cost, and in a restricted thermal package, such as in the Acorn Archimedes, while featuring in the Super Computer League tables, its initial, relatively, lower power and cooling implementation was soon adapted to embedded applications, such as laser printer raster image processing.^[40] Acorn, in partnership with Apple Inc, and VLSI, creating ARM Ltd, in 1990, to share R&D costs and find new markets for the ISA, who in partnership with TI, GEC, Sharp, Nokia, Oracle and Digital would develop low-power and embedded RISC designs, and target those market segments, which at the time were niche. With the rise in mobile, automotive, streaming, smart device computing, ARM became the most widely used ISA, the company estimating almost half of all CPUs shipped in history have been ARM.^[41]

Characteristics and design philosophy

Confusion around the definition of RISC deriving from the formulation of the term, along with the tendency to opportunistically categorize processor architectures with relatively few instructions (or groups of instructions) as RISC architectures, led to attempts to define RISC as a design philosophy. One attempt to do so was expressed as the following:

A RISC processor has an instruction set that is designed for efficient execution by a pipelined processor and for code generation by an optimizing compiler.

—Michael Slater, Microprocessor Report^[42]

Instruction set philosophy

A common misunderstanding of the phrase "reduced instruction set computer" is that instructions are simply eliminated, resulting in a smaller set of instructions.^[43] In fact, over the years, RISC instruction sets have grown in size, and today many of them have a larger set of instructions than many CISC CPUs.^{[44][45]} Some RISC processors such as the PowerPC have instruction sets as large as the CISC IBM System/370, for example; conversely, the DEC PDP-8—clearly a CISC CPU because many of its instructions involve multiple memory accesses—has only 8 basic instructions and a few extended instructions.^[46] The term "reduced" in that phrase was intended to describe the fact that the amount of work any single instruction accomplishes is reduced—at most a single data memory cycle—compared to the "complex instructions" of CISC CPUs that may require dozens of data memory cycles in order to execute a single instruction.^[47]

The term load-store architecture is sometimes preferred.

Another way of looking at the RISC/CISC debate is to consider what is exposed to the compiler. In a CISC processor, the hardware may internally use registers and flag bit in order to implement a single complex instruction such as `STRING MOVE`, but hide those details from the compiler. The internal operations of a RISC processor are "exposed to the compiler", leading to the backronym 'Relegate Interesting Stuff to the Compiler'.^{[48][49]}

Instruction format

Most RISC architectures have fixed-length instructions and a simple encoding, which simplifies fetch, decode, and issue logic considerably. This is among the main goals of the RISC approach.^[22]

Some of this is possible only due to the contemporary move to 32-bit formats. For instance, in a typical program, over 30% of all the numeric constants are either 0 or 1, 95% will fit in one byte, and 99% in a 16-bit value.^[50] When computers were based on 8- or 16-bit words, it would be difficult to have an immediate combined with the opcode in a single memory word, although certain instructions like increment and decrement did this implicitly by using a different opcode. In contrast, a 32-bit machine has ample room to encode an immediate value, and doing so avoids the need to do a second memory read to pick up the value. This is why many RISC processors allow a 12- or 13-bit constant to be encoded directly into the instruction word.^[22]

Assuming a 13-bit constant area, as is the case in the MIPS and RISC designs, another 19 bits are available for the instruction encoding. This leaves ample room to indicate both the opcode and one or two registers. Register-to-register operations, mostly math and logic, require enough bits to encode the two or three registers being used. Most processors use the three-operand format $A = B + C$, in which case three register numbers are needed. If the processor has 32 registers, each one requires a 5-bit number for 15 bits total. If one of these registers is replaced by an immediate, there is still room to encode the two remaining registers and the opcode. Common instructions found in multi-word systems, like INC and DEC, which reduce the number of words that have to be read before performing the instruction, are unnecessary in RISC as they can be accomplished with a single register and the immediate value 1.^[22]

The original RISC-I format remains a canonical example of the concept. It uses 7 bits for the opcode and a 1-bit flag for conditional codes, the following 5 bits for the destination register, and the next five for the first operand. This leaves 14 bits, the first of which indicates whether the following 13 contain an immediate value or uses only five of them to indicate a register for the second operand.^[22] A more complex example is the MIPS encoding, which used only 6 bits for the opcode, followed by two 5-bit registers. The remaining 16 bits could be used in two ways, one as a 16-bit immediate value, or as a 5-bit shift value (used only in shift operations, otherwise zero) and the remaining 6 bits as an extension on the opcode. In the case of register-to-register arithmetic operations, the opcode was 0 and the last 6 bits contained the actual code; those that used an immediate value used the normal opcode field at the front.^[51]

One drawback of 32-bit instructions is reduced code density, which is more adverse a characteristic in embedded computing than it is in the workstation and server markets RISC architectures were originally designed to serve. To address this problem, several architectures, such as SuperH (1992), ARM thumb (1994),^[52] MIPS16e (2004), Power Variable Length Encoding ISA (2006), RISC-V, and the Adapteva Epiphany, have an optional short, feature-reduced compressed instruction set. Generally, these instructions expose a smaller number of registers and fewer bits for immediate values, and often use a two-operand format to eliminate one register number from instructions. A two-operand format in a system with 16 registers requires 8 bits for register numbers, leaving another 8 for an opcode or other uses. The SH5 also follows this pattern, albeit having evolved in the opposite direction, having added longer 32-bit instructions to an original 16-bit encoding.

Hardware utilization

The most characteristic aspect of RISC is executing at least one instruction per cycle.^[35] Single-cycle operation is described as "the rapid execution of simple functions that dominate a computer's instruction stream", thus seeking to deliver an average throughput approaching one instruction per cycle for any single instruction stream.^[53]

Other features of RISC architectures include:

- Far fewer transistors dedicated to the core logic, which originally allowed designers to increase the size of the register set and increase internal parallelism.
- Uniform instruction format, using single word with the opcode in the same bit positions for simpler decoding
- All general-purpose registers can be used equally as source/destination in all instructions, simplifying compiler design (floating-point registers are often kept separate)
- Simple addressing modes with complex addressing performed by instruction sequences
- Few data types in hardware (no byte string or binary-coded decimal [BCD], for example)

RISC designs are also more likely to feature a Harvard memory model, where the instruction stream and the data stream are conceptually separated; this means that modifying the memory where code is held might not have any effect on the instructions executed by the processor (because the CPU has a separate instruction and data cache), at least until a special synchronization instruction is issued; CISC processors that have separate instruction and data caches generally keep them synchronized automatically, for backwards compatibility with older processors.

Many early RISC designs also shared the characteristic of having a branch delay slot, an instruction space immediately following a jump or branch. The instruction in this space is executed, whether or not the branch is taken (in other words the effect of the branch is delayed). This instruction keeps the ALU of the CPU busy for the extra time normally needed to perform a branch. Nowadays the branch delay slot is considered an unfortunate side effect of a particular strategy for implementing some RISC designs, and modern RISC designs generally do away with it (such as PowerPC and more recent versions of SPARC and MIPS).

Some aspects attributed to the first RISC-*labeled* designs around 1975 include the observations that the memory-restricted compilers of the time were often unable to take advantage of features intended to facilitate *manual* assembly coding, and that complex addressing modes take many cycles to perform due to the required additional memory accesses. It was argued that such functions would be better performed by sequences of simpler instructions if this could yield implementations small enough to leave room for many registers, reducing the number of slow memory accesses. In these simple designs, most instructions are of uniform length and similar structure, arithmetic operations are restricted to CPU registers and only separate *load* and *store* instructions access memory. These properties enable a better balancing of pipeline stages than before, making RISC pipelines significantly more efficient and allowing higher clock frequencies.

Yet another impetus of both RISC and other designs came from practical measurements on real-world programs. Andrew Tanenbaum summed up many of these, demonstrating that processors often had oversized immediates. For instance, he showed that 98% of all the constants in a program would fit in 13 bits, yet many CPU designs dedicated 16 or 32 bits to store them. This suggests that, to reduce the number of memory accesses, a fixed length machine could store constants in unused bits of the

instruction word itself, so that they would be immediately ready when the CPU needs them (much like immediate addressing in a conventional design). This required small opcodes in order to leave room for a reasonably sized constant in a 32-bit instruction word.

Since many real-world programs spend most of their time executing simple operations, some researchers decided to focus on making those operations as fast as possible. The clock rate of a CPU is limited by the time it takes to execute the slowest *sub-operation* of any instruction; decreasing that cycle-time often accelerates the execution of other instructions.^[54] The focus on "reduced instructions" led to the resulting machine being called a "reduced instruction set computer" (RISC). The goal was to make instructions so simple that they could *easily* be pipelined, in order to achieve a *single clock* throughput at *high frequencies*. This contrasted with CISC designs whose "crucial arithmetic operations and register transfers" were considered difficult to pipeline.^[55]

Later, it was noted that one of the most significant characteristics of RISC processors was that external memory was only accessible by a *load* or *store* instruction. All other instructions were limited to internal registers. This simplified many aspects of processor design: allowing instructions to be fixed-length, simplifying pipelines, and isolating the logic for dealing with the delay in completing a memory access (cache miss, etc.) to only two instructions. This led to RISC designs being referred to as *load–store* architectures.^[56]

Comparison to other architectures

Some CPUs have been specifically designed to have a very small set of instructions—but these designs are very different from classic RISC designs, so they have been given other names such as minimal instruction set computer (MISC) or transport triggered architecture (TTA).

RISC architectures have traditionally had few successes in the desktop PC and commodity server markets, where the x86-based platforms remain the dominant processor architecture. However, this may change, as ARM-based processors are being developed for higher performance systems.^[57] Manufacturers including Cavium, AMD, and Qualcomm have released server processors based on the ARM architecture.^{[58][59]} ARM further partnered with Cray in 2017 to produce an ARM-based supercomputer.^[60] On the desktop, Microsoft announced that it planned to support the PC version of Windows 10 on Qualcomm Snapdragon-based devices in 2017 as part of its partnership with Qualcomm. These devices will support Windows applications compiled for 32-bit x86 via an x86 processor emulator that translates 32-bit x86 code to ARM64 code.^{[61][62]} Apple announced they will transition their Mac desktop and laptop computers from Intel processors to internally developed ARM64-based SoCs called Apple silicon; the first such computers, using the Apple M1 processor, were released in November 2020.^[63] Macs with Apple silicon can run x86-64 binaries with Rosetta 2, an x86-64 to ARM64 translator.^[64]

Outside of the desktop arena, however, the ARM RISC architecture is in widespread use in smartphones, tablets and many forms of embedded devices. While early RISC designs differed significantly from contemporary CISC designs, by 2000 the highest-performing CPUs in the RISC line were almost indistinguishable from the highest-performing CPUs in the CISC line.^{[65][66][67]}

Use of RISC architectures

RISC architectures are now used across a range of platforms, from smartphones and tablet computers to some of the world's fastest supercomputers such as Fugaku, the fastest on the TOP500 list as of November 2020, and Summit, Sierra, and Sunway TaihuLight, the next three on that list.^[68]

Low-end and mobile systems

By the beginning of the 21st century, the majority of low-end and mobile systems relied on RISC architectures.^[69] Examples include:

- The ARM architecture dominates the market for low-power and low-cost embedded systems (typically 200–1800 MHz in 2014). It is used in a number of systems such as most Android-based systems, the Apple iPhone, iPod Touch, iPad, Apple Watch, and Apple TV, Palm, Microsoft Windows Phone (former Windows Mobile / Windows CE), RIM devices, Nintendo Game Boy Advance, DS, 3DS and Switch, Raspberry Pi, etc.
- IBM's PowerPC was used in the GameCube, Wii, PlayStation 3, Xbox 360 and Wii U gaming consoles.
- The MIPS line (at one point used in many SGI computers) was used in the PlayStation, PlayStation 2, Nintendo 64, PlayStation Portable game consoles, and residential gateways like Linksys WRT54G series.
- Hitachi's SuperH, originally in wide use in the 32X, Saturn, and Dreamcast consoles, now developed and sold by Renesas as the SH4.
- Atmel AVR, used in a variety of products ranging from Xbox handheld controllers and the Arduino open-source microcontroller platform to BMW cars.
- RISC-V, the current iteration of Berkeley's open standard RISC ISA, with 32- or 64-bit address spaces, a small core integer instruction set, and an experimental "Compressed" ISA for code density and designed for standard and special-purpose extensions.

Desktop and laptop computers

- IBM's PowerPC architecture was used in Apple's Macintosh computers from 1994, when they began a switch from Motorola 68000 family processors, to 2005, when they transitioned to Intel x86 processors.^[70]
- Some chromebooks use ARM-based platforms since 2012.^[71]
- Apple uses inhouse-designed processors based on the ARM architecture for its lineup of desktop and laptop computers since its transition from Intel processors,^[72] and the first such computers were released in November 2020.^[63]
- Microsoft uses Qualcomm^[73] ARM-based processors for its Surface line. HP Inc and Lenovo have released Windows PCs with an ARM-based Qualcomm Snapdragon.

Workstations, servers, and supercomputers

- MIPS, by Silicon Graphics (ceased making MIPS-based systems in 2006).
- SPARC, by Oracle (previously Sun Microsystems), and Fujitsu.
- IBM's IBM POWER architecture, PowerPC, and Power ISA were and are used in many of IBM's supercomputers, mid-range servers and workstations.

- Hewlett-Packard's PA-RISC, also known as HP-PA (discontinued at the end of 2008).
- Alpha, used in single-board computers, workstations, servers and supercomputers from Digital Equipment Corporation, then Compaq and finally Hewlett-Packard (HP)(discontinued as of 2007).
- RISC-V, the fifth Berkeley RISC ISA, with 64- or 128-bit address spaces, and the integer core extended with floating point, atomics and vector processing, and designed to be extended with instructions for networking, I/O, and data processing. A specification for a 64-bit superscalar design, "Rocket", is available for download. It is implemented in the European Processor Initiative processor.
- The ARM architecture currently in use by cloud providers for servers. One example is the AWS Graviton series processor used for various services on the AWS platform.^[74] ARM was also used in the Fujitsu A64FX chip to create Fugaku, the world's fastest supercomputer in 2020.

Open source, standard, or use

RISC architectures have become popular in open source processors and soft microprocessors since they are relatively simple to implement, which makes them suitable for FPGA implementations and prototyping, for instance. Examples include:

- OpenRISC, an open instruction set and micro-architecture first introduced in 2000.
- Open MIPS architecture, for part of 2019 the specifications were free to use, royalty free, for registered MIPS developers.^[75]
- OpenSPARC, in 2005, Sun released its Ultra Sparc documentation and specifications, under the GPLv2.
 - LEON, an open source, radiation-tolerant implementation of the SPARC V8 instruction set (targeting space applications).
- Libre-SOC, an open source SoC based on the Power ISA with extensions for video and 3D graphics.
- RISC-V, in 2010, the Berkeley RISC version 5, specification, tool chain, and brand, were made available, free of charge, for non-commercial purposes.^[76]
- SuperH - J Core, in 2015, a project to offer clean room implementations of the, patent expired, Hitachi SuperH RISC ISA was started.
- ARM DesignStart, in 2018 ARM, in partnership with FPGA supplier Xilinx, started to offer free access to some of ARM's IP, including FPGA specification for some older CPU cores.^[77]

Awards

In 2022 Steve Furber, John L. Hennessy, David A. Patterson and Sophie M. Wilson were awarded the Charles Stark Draper Prize by the United States National Academy of Engineering for their contributions to the invention, development, and implementation of reduced instruction set computer (RISC) chips.^{[78][79]}

See also

- Explicitly parallel instruction computing

- No instruction set computing
- One-instruction set computer
- Very long instruction word

References

1. Chen, Crystal; Novick, Greg; Shimano, Kirk. "Pipelining" (<https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html>). *RISC Architecture*.
2. Flynn, Michael J. (1995). *Computer Architecture: Pipelined and Parallel Processor Design*. Jones & Bartlett Learning. pp. 54–56. ISBN 0867202041.
3. Colwell, Robert P.; Hitchcock III, Charles Y.; Jensen, E. Douglas; Sprunt, H. M. Brinkley; Kollar, Charles P. (September 1985). "Instruction Sets and Beyond: Computers, Complexity, and Controversy" (https://pages.cs.wisc.edu/~markhill/restricted/ieeecomputer85_cisc.pdf) (PDF). *Computer*. IEEE. pp. 8–19.
4. Aletan, Samuel O. (1 April 1992). "An overview of RISC architecture" (<https://dl.acm.org/doi/10.1145/143559.143570>). *Proceedings of the 1992 ACM/SIGAPP Symposium on Applied computing: technological challenges of the 1990's*. SAC '92. Kansas City, Missouri: Association for Computing Machinery. pp. 11–20. doi:10.1145/143559.143570 (<https://doi.org/10.1145/143559.143570>).
5. Markoff, John (November 1984). "New Chips — RISC Chips" (<https://archive.org/details/byte-magazine-1984-11/page/n191/mode/2up>). *Byte*. Vol. 9, no. 12. McGraw-Hill. pp. 191–206.
6. Boursin de l'Arc, Philippe. "Histoire de l'Informatique et d'Internet" (<http://boursinp.free.fr/bonus4.htm>). *boursinp.free.fr*.
7. Manuel, Tom (3 September 1987). "Inside Technology — The Frantic Search for More Speed" (<https://worldradiohistory.com/Archive-Electronics/80s/87/Electronics-1987-09-03.pdf#page=65>) (PDF). *Electronics*. McGraw-Hill. pp. 59–62.
8. "Japan's Fugaku gains title as world's fastest supercomputer" (https://www.riken.jp/en/news_pubs/news/2020/20200623_1/). *RIKEN*. Retrieved 24 June 2020.
9. Fisher, Joseph A.; Faraboschi, Paolo; Young, Cliff (2005). *Embedded Computing: A VLIW Approach to Architecture, Compilers and Tools* (https://archive.org/details/embeddedcomputing00fish_183). Elsevier. p. 55 (https://archive.org/details/embeddedcomputing00fish_183/page/n91). ISBN 1558607668.
10. Reilly, Edwin D. (2003). *Milestones in computer science and information technology* (https://archive.org/details/milestonesincomp0000reil_m9h9). Greenwood Publishing. pp. 50 (https://archive.org/details/milestonesincomp0000reil_m9h9/page/50). ISBN 1-57356-521-0.
11. Grishman, Ralph (1974). *Assembly Language Programming for the Control Data 6000 Series and the Cyber 70 Series*. Algorithmics Press. p. 12. OCLC 425963232 (<https://search.worldcat.org/oclc/425963232>).
12. Dongarra, Jack J.; et al. (1987). *Numerical Linear Algebra on High-Performance Computers* (https://archive.org/details/numericallineara00dong_316). pp. 6 (https://archive.org/details/numericallineara00dong_316/page/n24). ISBN 0-89871-428-1.
13. Cocke, John; Markstein, Victoria (January 1990). "The evolution of RISC technology at IBM" (<https://www.cis.upenn.edu/~milom/cis501-Fall11/papers/cocke-RISC.pdf>) (PDF). *IBM Journal of Research and Development*. **34** (1): 4–11. doi:10.1147/rd.341.0004 (<https://doi.org/10.1147/rd.341.0004>).
14. *IBM System/370 System Summary* (<http://www.beagle-ears.com/lars/engineer/comphist/model360.htm>) (Technical report). IBM. January 1987.
15. Šilc, Jurij; Robič, Borut; Ungerer, Theo (1999). *Processor architecture: from dataflow to superscalar and beyond* (<https://archive.org/details/processorarchite00ilcd>). Springer. pp. 33 (<https://archive.org/details/processorarchite00ilcd/page/n52>). ISBN 3-540-64798-8.

16. *Funding a Revolution: Government Support for Computing Research* by Committee on Innovations in Computing and Communications 1999 ISBN 0-309-06278-0 page 239
17. Nurmi, Jari (2007). *Processor design: system-on-chip computing for ASICs and FPGAs* (<http://archive.org/details/processor designs00nurmi>). Springer. pp. 40 (<https://archive.org/detail/s/processor designs00nurmi/page/n56>)–43. ISBN 978-1-4020-5529-4.
18. Hill, Mark Donald; Jouppi, Norman Paul; Sohi, Gurindar (1999). *Readings in computer architecture*. Gulf Professional. pp. 252–4. ISBN 1-55860-539-8.
19. Starnes, Thomas (May 1983). "Design Philosophy Behind Motorola's MC68000" (<http://www.easy68k.com/paulrsm/doc/dpbm68k2.htm>). Byte. p. Photo 1.
20. Patterson, David (30 May 2018). "RISCy History" (<https://www.sigarch.org/riscy-history/>). AM SIGARCH.
21. "Example: Berkeley RISC II" (<https://web.archive.org/web/20220613053600/http://euler.mat.uson.mx/~havillam/ca/CS323/0708.cs-323005.html>). Archived from the original (<http://euler.mat.uson.mx/~havillam/ca/CS323/0708.cs-323005.html>) on 13 June 2022.
22. Patterson, David A.; Sequin, Carlo H. (1981). *RISC I: A Reduced Instruction Set VLSI Computer* (<http://dl.acm.org/citation.cfm?id=801895&CFID=730308602&CFTOKEN=83826969>). 8th annual symposium on Computer Architecture. Minneapolis, MN, USA. pp. 443–457. doi:10.1145/285930.285981 (<https://doi.org/10.1145%2F285930.285981>). As PDF (<https://web.archive.org/web/20170406022438/https://pdfs.semanticscholar.org/9a8a/859d1a9352e13e7fa4fc9e66f1de7066a9c9.pdf>)
23. Patterson, D. A.; Ditzel, D. R. (1980). "The case for the reduced instruction set computer". *ACM SIGARCH Computer Architecture News*. **8** (6): 25–33. CiteSeerX 10.1.1.68.9623 (<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.68.9623>). doi:10.1145/641914.641917 (<https://doi.org/10.1145%2F641914.641917>). S2CID 12034303 (<https://api.semanticscholar.org/CorpusID:12034303>).
24. Sequin, Carlo; Patterson, David (July 1982). *Design and Implementation of RISC I* (<http://www.eecs.berkeley.edu/Pubs/TechRpts/1982/CSD-82-106.pdf>) (PDF). Advanced Course on VLSI Architecture. University of Bristol. CSD-82-106.
25. Chow, Paul (1989). *The MIPS-X RISC microprocessor*. Springer. pp. xix–xx. ISBN 0-7923-9045-8.
26. Nurmi 2007, pp. 52–53
27. Weaver, Vincent; McKee, Sally. *Code Density Concerns for New Architectures* (http://web.ee.ce.maine.edu/~vweaver/papers/iccd09/iccd09_density.pdf) (PDF). ICCD 2009.
28. Waterman, Andrew; Lee, Yunsup; Patterson, David A.; Asanovi, Krste. "The RISC-V Instruction Set Manual, Volume I: Base User-Level ISA version 2.0" (<https://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-54.html>). University of California, Berkeley. Technical Report EECS-2014-54. Retrieved 1 March 2022.
29. "Acorn Computers Announce The Availability Of An Evaluation System For Their Single Chip, 32-Bit Reduced Instruction Set Computer - The Arm" (https://chrisacorns.computinghistory.org.uk/docs/Acorn/PR/Acorn_announces_the_ARM_Evaluation_kit.pdf) (PDF) (Press release). Acorn Computers Limited. 7 July 1986. Retrieved 29 June 2025.
30. Garnsey, Elizabeth; Lorenzoni, Gianni; Ferriani, Simone (March 2008). "Speciation through entrepreneurial spin-off: The Acorn-ARM story" (<http://www2.sa.unibo.it/~simone.ferriani/Download/Speciation%20through%20Entrepreneurial%20Spin-off.pdf>) (PDF). *Research Policy*. **37** (2): 210–224. doi:10.1016/j.respol.2007.11.006 (<https://doi.org/10.1016%2Fj.respol.2007.11.006>). S2CID 73520408 (<https://api.semanticscholar.org/CorpusID:73520408>). Retrieved 2 June 2011. "[...] the first silicon was run on April 26th 1985."
31. Connolly, James (12 May 1986). "MIPS Computer Systems jumps into OEM mart with RISC-based products" (<https://archive.org/details/computerworld2019unse/page/34/mode/1up>). *Computerworld*. p. 34. Retrieved 29 June 2025.

32. Fotland, David A.; Shelton, John F.; Bryg, William R.; La Fetra, Ross V.; Boschma, Simin I.; Yeh, Allan S.; Jacobs, Edward M. (March 1987). "Hardware Design of the First HP Precision Architecture Computers" (https://archive.org/details/Hewlett-Packard_Journal_Vol._38_No._3_1987-03_Hewlett-Packard/page/n3/mode/2up). *Hewlett-Packard Journal*. **38** (3): 6. Retrieved 6 October 2020.
33. Tucker, Allen B. (2004). *Computer science handbook* (<https://archive.org/details/computersciencee00tuck>). Taylor & Francis. pp. 100 (<https://archive.org/details/computersciencee00tuck/page/n1244>)–6. ISBN 1-58488-360-X.
34. Crawford, John; Hill, Gene; Leukhardt, Jill; Prak, Jan Willem; Slager, Jim. "Intel 386 Microprocessor Design and Development Oral History Panel" (<https://archive.computerhistory.org/resources/access/text/2015/06/102702019-05-01-acc.pdf>) (PDF) (Interview). Interviewed by Jim Jarrett. Mountain View, California: Computer History Museum. Retrieved 15 May 2025.
35. Marshall, Trevor; Tazelaar, Jane Morrill (February 1989). "Worth the RISC" (https://archive.org/details/eu_BYTE-1989-02_OCR/page/n299/mode/2up?view=theater). *BYTE*. pp. 245–249. Retrieved 8 October 2024.
36. Bell, Gordon; Strecker, W.D. What Have We Learned from the PDP-11 - What We Have Learned from VAX and Alpha (https://archive.computerhistory.org/resources/text/DEC/alpha/dec.alpha.bell_stecker.pdp-11_vax_and_alpha.1998.102630388.pdf) (PDF) (Report). Retrieved 26 June 2025.
37. "IBM's Born-Again Workstation Line Will Exorcise RT Ghost, Users Say" (<https://books.google.com/books?id=gTAEAAAAMBAJ&pg=PA1>). *InfoWorld*. Vol. 12, no. 8. 19 February 1990. pp. 1, 101. Retrieved 28 June 2025.
38. Furlong, Thomas C.; Nielsen, Michael J. K.; Wilhelm, Neil C. (Spring 1990). "Development of the DECstation 3100" (http://www.bitsavers.org/pdf/dec/dtj/dtj_v02-02_1990.pdf) (PDF). *Digital Technical Journal*. **2** (2). Digital Equipment Corporation.
39. Fisher, Susan E. (9 November 1992). "It's show time for RISC; DEC to debut Alpha systems, as Sun, HP wait in wings" (<https://galeapps.gale.com/apps/auth?userGroupName=wikipedia&sid=bookmark-GPS&origURL=https%3A%2F%2Fgo.gale.com%2Fps%2Fi.do%3Fp%3DGPS%26u%3Dwikipedia%26id%3DGALE%25257CA12756142%26v%3D2.1%26it%3Dr%26sid%3Dbookmark-GPS%26asid%3D940f7d98&prodId=GPS>). *PC Week*. Vol. 9, no. 45. Retrieved 29 September 2025 – via Gale.
40. "Olivetti buys RISC card" (<https://archive.org/details/AcornUser073-Aug88/page/n8/mode/1up>). *Acorn User*. August 1988. p. 7. Retrieved 24 May 2021.
41. Bradshaw, Tim (23 August 2023). "Arm searches for growth beyond smartphones" (<https://www.ft.com/content/0139781c-1db5-4104-bf4f-fdc087971020>). *www.ft.com*. Retrieved 23 June 2024.
42. Slater, Michael (June 1990). "What is RISC?" (https://archive.org/details/ieee_micro_v10n3_june_90/page/n99/mode/1up). *IEEE Micro*. pp. 96–95. Retrieved 20 March 2023.
43. Esponda, Margarita; Rojas, Ra'ul (September 1991). "Section 2: The confusion around the RISC concept" (<http://www.inf.fu-berlin.de/lehre/WS94/RA/RISC-9.html>). *The RISC Concept — A Survey of Implementations*. Freie Universitat Berlin. B-91-12.
44. Stokes, Jon "Hannibal". "RISC vs. CISC: the Post-RISC Era" (<https://arstechnica.com/cpu/4q99/risc-cisc/rvc-5.html#Branch>). *Ars Technica*.
45. Borrett, Lloyd (June 1991). "RISC versus CISC" (<http://www.borrett.id.au/computing/art-1991-06-02.htm>). *Australian Personal Computer*.
46. Jones, Douglas W. "Doug Jones's DEC PDP-8 FAQs" (<http://homepage.cs.uiowa.edu/~jones/pdp8/faqs/#instrs>). *PDP-8 Collection, The University Of Iowa Department of Computer Science*.

47. Dandamudi, Sivarama P. (2005). "Ch. 3: RISC Principles". *Guide to RISC Processors for Programmers and Engineers* (https://archive.org/details/guidetoriscproce00dand_422). Springer. pp. 39 (https://archive.org/details/guidetoriscproce00dand_422/page/n49)–44. doi:10.1007/0-387-27446-4_3 (https://doi.org/10.1007%2F0-387-27446-4_3). ISBN 978-0-387-21017-9. "the main goal was not to reduce the number of instructions, but the complexity"
48. Walls, Colin (18 April 2016). "CISC and RISC" (<https://blogs.sw.siemens.com/embedded-software/2016/04/18/cisc-and-risc>).
49. Fisher, Joseph A.; Faraboschi, Paolo; Young, Cliff (2005). *Embedded Computing: A VLIW Approach to Architecture, Compilers and Tools* (<https://books.google.com/books?id=R5UXl6Jo0XYC>). Elsevier. p. 57. ISBN 9781558607668.
50. Alexander, W. Gregg; Wortman, David (November 1975). "Static and Dynamic Characteristics of XPL Programs". *IEEE Computer*. **8** (11): 41–48. doi:10.1109/C-M.1975.218804 (<https://doi.org/10.1109%2FC-M.1975.218804>). S2CID 39685209 (<https://api.semanticscholar.org/CorpusID:39685209>).
51. Soares, João; Rocha, Ricardo. "Encoding MIPS Instructions" (https://www.dcc.fc.up.pt/~ricroc/aulas/1920/ac/apontamentos/P04_encoding_mips_instructions.pdf) (PDF).
52. Staff, Embedded (24 September 2003). "ARM Thumb Instruction Set" (<https://www.embedded.com/introduction-to-arm-thumb/>). *Embedded.com*. Retrieved 23 June 2024.
53. Colwell, Robert P.; Hitchcock III, Charles Y.; Jensen, E. Douglas; Sprunt, H. M. Brinkley; Kollar, Charles P. (September 1985). "Instruction Sets and Beyond: Computers, Complexity and Controversy" (https://archive.org/details/tutorialsoftware0000unse_s5q2/page/116/mode/2up). *Computer*. **18** (9). The Institute of Electrical and Electronics Engineers, Inc.: 8–19. doi:10.1109/MC.1985.1663000 (<https://doi.org/10.1109%2FMC.1985.1663000>). S2CID 7306378 (<https://api.semanticscholar.org/CorpusID:7306378>). Retrieved 25 March 2023.
54. "Microprocessors From the Programmer's Perspective" (<http://www.ddj.com/architect/184408418>) by Andrew Schulman 1990
55. Sweetman, Dominic (1987). "The RISC Workstation" (<https://archive.org/details/internationalcom0000yvon/page/83/mode/1up>). *Proceedings of the Conference Held at Computer Graphics 87, London, October 1987*. Online Publications. pp. 83–91. ISBN 0-86353-092-3. Retrieved 26 March 2024.
56. Dowd, Kevin; Loukides, Michael K. (1993). *High Performance Computing* (<https://archive.org/details/highperformanccec00dowd>). O'Reilly. ISBN 1565920325.
57. Vincent, James (9 March 2017). "Microsoft unveils new ARM server designs, threatening Intel's dominance" (<https://www.theverge.com/2017/3/9/14867310/arm-servers-microsoft-intel-compute-conference>). *The Verge*. Retrieved 12 May 2017.
58. Russell, John (31 May 2016). "Cavium Unveils ThunderX2 Plans, Reports ARM Traction is Growing" (<https://www.hpcwire.com/2016/05/31/cavium-unveils-thunderx2-plans-reports-arm-traction-growing/>). HPC Wire. Retrieved 8 March 2017.
59. *AMD's first ARM-based processor, the Opteron A1100, is finally here* (<https://www.extremetech.com/extreme/221282-amds-first-arm-based-processor-the-opteron-a1100-is-finally-here>), ExtremeTech, 14 January 2016, retrieved 14 August 2016
60. Feldman, Michael (18 January 2017). "Cray to Deliver ARM-Powered Supercomputer to UK Consortium" (<https://www.top500.org/news/cray-to-deliver-arm-powered-supercomputer-to-uk-consortium/>). Top500.org. Retrieved 12 May 2017.
61. "Microsoft is bringing Windows desktop apps to mobile ARM processors" (<https://www.theverge.com/2016/12/7/13866936/microsoft-windows-10-arm-desktop-apps-support-qualcomm>). *The Verge*. 8 December 2016. Retrieved 8 December 2016.
62. "How x86 emulation works on ARM" (<https://learn.microsoft.com/en-us/windows/arm/apps-on-arm-x86-emulation>). *Microsoft Docs*. 23 October 2023. Retrieved 27 October 2023.

63. "Introducing the next generation of Mac" (<https://www.apple.com/newsroom/2020/11/introducing-the-next-generation-of-mac/>) (Press release). Apple Inc. 10 November 2020.
64. "macOS Big Sur is here" (<https://www.apple.com/newsroom/2020/11/macOS-big-sur-is-here/>) (Press release). Apple Inc. 12 November 2020.
65. Carter, Nicholas P. (2002). *Schaum's Outline of Computer Architecture* (<https://books.google.com/books?id=24V00tD7HeAC&pg=PT105>). McGraw Hill Professional. p. 96. ISBN 0-07-136207-X.
66. Jones, Douglas L. (2000). "CISC, RISC, and DSP Microprocessors" (<https://web.archive.org/web/20080907083430/http://www.ifp.uiuc.edu/~jones/RISCvCISCvDSP.pdf>) (PDF). Archived from the original (<http://www.ifp.uiuc.edu/~jones/RISCvCISCvDSP.pdf>) (PDF) on 7 September 2008. Retrieved 17 December 2008.
67. Singh, Amit. "A History of Apple's Operating Systems" (<https://web.archive.org/web/20200403101510/http://www.kernelthread.com/publications/appleoshistory/5.html>). Archived from the original (<http://www.kernelthread.com/publications/appleoshistory/5.html>) on 3 April 2020. "the line between RISC and CISC has been growing fuzzier over the years"
68. "Top 500 The List: November 2020" (<https://www.top500.org/lists/2020/11/>). *TOP 500*. Retrieved 2 January 2021.
69. Dandamudi 2005, pp. 121–123
70. Bennett, Amy (2005). "Apple shifting from PowerPC to Intel" (<https://web.archive.org/web/20201113060929/https://www.computerworld.com/article/2809210/apple-shifting-from-powerpc-to-intel.html>). *Computerworld*. Archived from the original (<https://www.computerworld.com/article/2809210/apple-shifting-from-powerpc-to-intel.html>) on 13 November 2020. Retrieved 24 August 2020.
71. Vaughan-Nichols, Steven J. "Review: The ARM-powered Samsung Chromebook" (<https://www.zdnet.com/article/review-the-arm-powered-samsung-chromebook/>). *ZDNet*. Retrieved 28 April 2021.
72. DeAngelis, Marc (22 June 2020). "Apple starts its two-year transition to ARM this week" (<https://www.engadget.com/apple-arm-transition-timeline-191106454.html>). *Engadget*. Retrieved 24 August 2020. "Apple has officially announced that it will be switching from Intel processors to its own ARM-based, A-series chips in its Mac computers."
73. Smith, Chris (16 September 2020). "Microsoft to launch a new ARM-based Surface this fall" (<https://bgr.com/tech/surface-pro-x-2020-release-date-october-snapdragon-8cx-gen-2-chip-5872000/>). *Boy Genius Report*. Retrieved 20 March 2023.
74. "ARM Processor - AWS Graviton Processor - AWS" (<https://aws.amazon.com/ec2/graviton/>). *Amazon Web Services, Inc.* Retrieved 9 January 2024.
75. "Wave Computing Shuttles MIPS Open Programme with Immediate Effect" (<https://abopen.com/news/wave-computing-shuttles-mips-open-programme-with-immediate-effect/>). *AB Open*. 15 November 2019. Retrieved 23 June 2024.
76. "Branding Guidelines – RISC-V International" (<https://riscv.org/about/risc-v-branding-guidelines/>). Retrieved 23 June 2024.
77. "Arm Expands Design Possibilities with Free Cortex-M Processors for Xilinx FPGAs" (<https://newsroom.arm.com/news/arm-expands-design-possibilities-with-free-cortex-m-processors-for-xilinx-fpgas>). *Arm Newsroom*. Retrieved 23 June 2024.
78. "Recipients of the Charles Stark Draper Prize for Engineering" (<https://www.nae.edu/55291/DraperWinners>). *nae.edu*.
79. "Charles Stark Draper Prize for Engineering" (<https://www.nae.edu/20681/DraperPrize>). *nae.edu*.

External links

- "RISC vs. CISC" (<http://cs.stanford.edu/people/eroberts/courses/soco/projects/2000-01/risc/riscisc/>). *RISC Architecture*. Stanford University. 2000.
- "What is RISC" (<http://cs.stanford.edu/people/eroberts/courses/soco/projects/2000-01/risc/whatis/>). *RISC Architecture*. Stanford University. 2000.
- Savard, John J. G. "Not Quite RISC" (<http://www.quadibloc.com/arch/sriscint.htm>). *Computers*.
- Mashey, John R. (5 September 2000). "Yet Another Post of the Old RISC Post [unchanged from last time]" (<https://groups.google.com/d/topic/comp.arch/IZP5KUJprHw/discussion>). Newsgroup: comp.arch (news:comp.arch). Usenet: 8p20b0\$dhh\$3@murrow.corp.sgi.com (news:8p20b0\$dhh\$3@murrow.corp.sgi.com). "Nth re-posting of CISC vs RISC (or what is RISC, really)"

Retrieved from "https://en.wikipedia.org/w/index.php?title=Reduced_instruction_set_computer&oldid=1313977856"